# SMART LENDER

GitHub: github.com/manikanta-00/Smart_Lender

## Applicant Credibility Prediction for Loan Approval

Using Machine Learning

# Project Documentation

Category: Machine Learning

**Team Lead:** Balaji Patheti

**Team Members**: Babikiran Aveti,
Jagadeesh Siddhardha Bera,
Venkat Mounish Reddy Muddam,
Manikanta Velpula

# Table of Contents

# 1. Project Title

Smart Lender – Applicant Credibility Prediction for Loan Approval Using Machine Learning

# 2. Abstract

Credit risk evaluation is one of the most important factors influencing a country's financial stability, as it is the core mechanism through which banks manage lending risk. Manually evaluating every loan applicant is slow, inconsistent, and prone to human bias. This project, Smart Lender, applies supervised Machine Learning classification techniques to automatically predict whether a loan applicant is likely to be approved, based on demographic and financial attributes such as income, credit history, employment status, and property area. The trained model is deployed through a Flask-based web application, allowing users to receive instant, data-driven loan eligibility predictions through a simple browser interface.

# 3. Problem Statement

Financial institutions receive a high volume of loan applications, and assessing each applicant's creditworthiness manually is both time-consuming and inconsistent across evaluators. Incorrect approvals increase the bank's risk of default, while incorrect rejections deny credit to genuinely eligible applicants. There is a need for a reliable, automated, and repeatable system that can analyze historical loan data, learn the underlying approval patterns, and predict the eligibility of new applicants accurately and instantly.

# 4. Features

- Predicts loan approval status (Approved / Not Approved) instantly based on 11 applicant parameters.
- Clean, responsive web interface built with Flask and HTML/CSS — no technical knowledge required to use it.
- Multiple classification algorithms (Decision Tree, Random Forest, KNN) trained and compared to select the best-performing model.
- Robust data preprocessing pipeline: missing-value handling, categorical encoding, and class-imbalance handling.

- Trained model persisted with Pickle for fast, repeatable predictions without retraining.
- Simple three-page flow — Home → Application Form → Result — for a smooth user experience.
- Lightweight and easy to run locally or deploy to any Flask-compatible hosting platform.

# 5. Tech Stack

| Layer | Technology Used |
|---|---|
| Programming Language | Python 3 |
| Data Handling | pandas, numpy |
| Machine Learning | scikit-learn |
| Model Persistence | pickle |
| Web Framework | Flask |
| Frontend | HTML5, CSS3 |
| IDE | VS Code / Jupyter Notebook |
| Version Control | Git & GitHub |

# 6. Dataset & Model Summary

The dataset used, loan_prediction.csv, contains 614 records with 13 attributes describing each loan applicant, including Gender, Marital Status, Dependents, Education, Employment type, Applicant and Co-applicant Income, Loan Amount, Loan Term, Credit History, Property Area, and the final Loan Status (target variable).

## 6.1 Data Preprocessing

- Missing categorical values (Gender, Married, Dependents, Self_Employed, Credit_History) were imputed using the mode of each column.
- Missing numerical values (LoanAmount, Loan_Amount_Term) were imputed using the mean/mode as appropriate.
- Categorical variables were encoded to numeric form (e.g., Gender: Male=0/Female=1, Property_Area: Urban=2/Semiurban=1/Rural=0).
- Class imbalance in the target variable was addressed using class-weighted training to prevent bias toward the majority class.
- The cleaned dataset was split into training (80%) and testing (20%) subsets for model evaluation.

## 6.2 Model Building & Evaluation

Three classification algorithms were trained and evaluated on the preprocessed dataset: Decision Tree, Random Forest, and K-Nearest Neighbors (KNN). Each model was evaluated using accuracy and F1-score on a held-out test set.

| Model | Accuracy | F1-Score | Notes |
|---|---|---|---|
| Decision Tree | 73.98% | 0.805 | Prone to overfitting on small data |
| Random Forest | 82.11% | 0.876 | Selected as final model — robust & stable |
| K-Nearest Neighbors | 84.55% | 0.897 | Requires feature scaling; more sensitive to input ranges |

The Random Forest Classifier was selected as the final production model. Although KNN scored marginally higher in this test split, Random Forest was chosen for its robustness, lower sensitivity to feature scaling, and stability across different input distributions, making it better suited for a simple deployed web application. The final model was trained on the complete dataset and serialized as rdf.pkl using pickle.

## 6.3 System Architecture

The application follows a simple client-server architecture:

- User submits loan details through the web form (input.html).
- Flask backend (app.py) receives the form data via a POST request.
- Input data is structured into a DataFrame matching the model's expected feature order.
- The trained Random Forest model (rdf.pkl) predicts the loan status.
- The result is rendered back to the user on the output page (output.html).

# 7. Installation Steps

Step 1: Clone the repository

git clone https://github.com/manikanta-00/Smart_Lender.git

Step 2: Navigate into the project folder

cd Smart_Lender/SmartLender

Step 3: Create and activate a virtual

environment python -m venv venv

Step 4: Install dependencies

pip install -r requirements.txt

Step 5: Run the application

python app.py

Step 6: Open in browser at http://127.0.0.1:5000

# 8. Usage Instructions

**Step 1:** On the home page, click the Predict button to open the loan application form.

**Step 2:** Fill in the required details: Gender, Marital Status, Dependents, Education, Employment Status, Applicant & Co-applicant Income, Loan Amount, Loan Term, Credit History, and Property Area.

**Step 3:** Click Submit to send the form data to the backend.

**Step 4:** The Flask server processes the input through the trained Random Forest model and instantly returns the prediction — Loan will be Approved or Loan will Not be Approved.

**Step 5:** Click "Try Another" on the result page to submit a new application.

# 9. Screenshots

## 9.1 Loan Application Form


*Figure 1: Loan application input form*

## 9.2 Financial Details Section


*Figure 2: Financial details and submission*

## 9.3 Landing Page

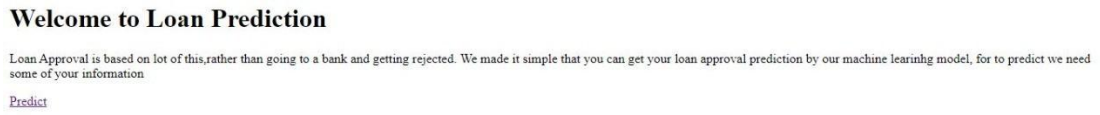


*Figure 3: Home page of the Smart Lender application*

# 10. Future Scope

- Integrate advanced ensemble models such as XGBoost or LightGBM for improved predictive accuracy.
- Deploy the application on a cloud platform (e.g., Render, AWS, IBM Cloud) for public accessibility.
- Add explainability (e.g., SHAP/LIME) so applicants understand the reasoning behind a decision.
- Introduce a database to log applications and predictions for audit and analytics.
- Add user authentication for banks/admins to manage applications.

# 11. Team Member Details

| Name | Role |
|---|---|
| Balaji Patheti | Team Lead |
| Babikiran Aveti | Member |
| Jagadeesh Siddhardha Bera | Member |
| Venkat Mounish Reddy Muddam | Member |
| Manikanta Velpula | Member |

# 12. Conclusion

Smart Lender demonstrates a complete, end-to-end machine learning pipeline — from raw data preprocessing and model training to deployment via a functional web application. The project achieves reliable loan approval predictions using a Random Forest Classifier and provides a practical, user-friendly interface, illustrating how machine learning can support and accelerate real-world financial decision-making.